Lab 10 Answers

Yanqiao Chen*
Department of Computer Science and Engineering
Southern University of Science and Technology
12412115@mail.sustech.edu.cn

May 15, 2026

1 Question 1

We show that $\overline{EQ_{CFG}}$ is Turing-recognizable. We construct the following recognizer.

$D =$ "On input $\langle G_1, G_2 \rangle$, where G_1 and G_2 are CFGs:

- Enumerate all strings in Σ^* in lexicographic order, and for each string w, do the following:
  - Run the Turing machine for A_{CFG} on input $\langle G_1, w \rangle$ and the Turing machine for A_{CFG} on input $\langle G_2, w \rangle$ in parallel.
  - If one of the two Turing machines accepts and the other rejects, then accept.

If they are different, then there must exist a string w that is generated by one of the two CFGs but not the other. In this case, D will accept. If the CFGs are the same, then they may loop or reject, but they will never accept. Therefore, D is a recognizer for $\overline{EQ_{CFG}}$.

Thus, $\overline{EQ_{CFG}}$ is Turing-recognizable, thus EQ_{CFG} is co-Turing-recognizable.

2 Question 2

No, it does not. Let $f(x) = \begin{cases} 1, & x \in A \\ 0, & x \notin A \end{cases}$. Thus let $B = \{1\}$. Then f is a reduction from A to B. However, let $A = \{1^k 0^k \mid k \geq 0\}$, which is not regular, and $B = \{1\}$, which is regular. Therefore, A is not regular, but B is regular. Thus, the regularity of B does not imply the regularity of A.

3 Question 3

We know A is decidable $\iff$ A is Turing-recognizable and $\overline{A}$ is Turing-recognizable. Since A is Turing-recognizable, we only need to show that $\overline{A}$ is Turing-recognizable and $A \leq_m \overline{A}$. Thus, $f(x) \in \overline{A}$ if and only if $x \in A$ and

$f(x) \in A$ if and only if $x \in \overline{A}$. We construct the following Turing machine to recognize $\overline{A}$.

$$D = \text{“On input } w\text{, where } w \text{ is a string:}$$

- For any input, D run the Turing machine to map w to $f(w)$, and then run the Turing machine for A on input $f(w)$.
- If the Turing machine for A accepts, we accept.

Thus, $\overline{A}$ is Turing-recognizable. Therefore, A is decidable.

4 Question 4

Let M' be a Turing machine.

$$M' = \text{“On input } \langle x \rangle\text{, where } x \text{ is a string:}$$

- If $x = 01$, accept.
- Then run M on input w, if M accepts, then accept; if M rejects, then reject.

This construction perform a mapping reduction $f(\langle M, x \rangle) = \langle M' \rangle$. If the decider decides M' whether is accept w if and only if it accepts w^R, then if $x \neq 01$, then M' accepts x if and only if M accepts w, which means that it accept both w and w^R. Thus, if a decider can decide $M' \in T$, then it must decide whether M' accept w^R, which include 10, thus it must decide whether M accepts w, which is equivalent to deciding A_{TM}.

Thus, if M' is decidable, then the decider decides A_{TM}, which is a contradiction. Therefore, T is undecidable.

5 Question 5

We prove from right to left. If $A \leq_m A_{TM}$, we construct the following Turing machine to recognize A.

$$D = \text{“On input } w\text{, where } w \text{ is a string:}$$

- Run the Turing machine to map w to $f(w)$, and then run the Turing machine M on input $\langle M, f(w) \rangle$. Then run the Turing machine for A_{TM} on input $\langle M, f(w) \rangle$ to recognize whether M accepts $f(w)$.
- If the Turing machine for A_{TM} accepts, then accept; if it rejects, then reject.

We prove from left to right. If A is recognizable, then M recognizes A. Let a computable function f map w to $\langle M, w \rangle$, i.e., $f(w) = \langle M, w \rangle$. Thus $w \in A \iff \langle M, w \rangle \in A_{TM} \iff f(w) \in A_{TM}$. Thus, $A \leq_m A_{TM}$.

6 Question 6

Construct a mapping reduction from $HALT_{TM}$ to T.

$$f(\langle M, w\rangle) = \langle M'\rangle$$

where M' is a Turing machine defined as follows:

$$M' = \text{“On input } x\text{, where } x \text{ is a string:}$$

- If $x \neq \epsilon$, accept.
- Then run M on input w, if M halts on w, accept; otherwise, reject.

Thus, if M' can be decided, then we can decide whether M accepts w, which is equivalent to deciding $HALT_{TM}$. Therefore, T is undecidable.

7 Question 7

- Undecidable. "$L(M)$ is infinite" is non-trivial, since there exist both finite and infinite languages in the set of all languages, thus it is undecidable by Rice's theorem.
- Undecidable. "$1011 \in L(M)$" is non-trivial, since not all Turing machines accept this string, thus it is undecidable by Rice's theorem.
- Undecidable. "$L(M) = \Sigma^*$" is non-trivial, since not all Turing machines accept all strings, thus it is undecidable by Rice's theorem.

8 Question 8

- True.
- False.
- True.
- False.
- False.
- True.

9 Question 9

We may draw the CYK parsing table as follows.

As $S \in T_{1,4}$, w is generated by the CFG.

Table 1: CYK parsing table for $w = baba$

T	{R, T}	S	{S, R, T}
	R	S	S
		T	{R, T}
			R

10 Question 10

We now show that **P** is closed under union, concatenation and complementation.

- Union: Let $L_1, L_2 \in \mathbf{P}$. Then there exist polynomial-time Turing machines M_1, M_2 that decide L_1, L_2 respectively. We can construct a Turing machine M that decides $L_1 \cup L_2$ as follows: on input w, run M_1 and M_2 in parallel, and accept if either accepts.
- Concatenation: Let $L_1, L_2 \in \mathbf{P}$. Then there exist polynomial-time Turing machines M_1, M_2 that decide L_1, L_2 respectively. We can construct a Turing machine M that decides $L_1 \cdot L_2$ as follows: on input w, enumerate where is the split point and check if the prefix is in L_1 and the suffix is in L_2, both in polynomial time.
- Complementation: Let $L \in \mathbf{P}$. Then there exists a polynomial-time Turing machine M that decides L. We can construct a Turing machine M' that decides $\overline{L}$ as follows: on input w, run M on w, and flip the output.

11 Question 11

We use BFS algorithm to show that $CONNECTED = \{\langle G \rangle \mid G \text{ is a connected graph}\}$ is in **P**.

$M =$ "On input $\langle G \rangle$, where G is a graph:

- Run BFS on G starting from an arbitrary vertex v and mark all visited vertices.
- If all vertices are visited, then accept; otherwise, reject.

The BFS algorithm runs in polynomial time for graph with n nodes, thus $CONNECTED \in \mathbf{P}$.

12 Question 12

Construct a Turing machine that decides ALL_{DFA} in polynomial time. Take the complement of the DFA. Starting from the initial state, we perform a BFS to check whether there exists a path from the initial state to an accept state. If such a path exists, then we reject; otherwise, we accept. The BFS algorithm runs in polynomial time for DFA with n states, thus $ALL_{DFA} \in \mathbf{P}$.

If there is a path, then $\overline{L(M)} \neq \emptyset$, thus $L(M) \neq \Sigma^*$. If there is no path, then $\overline{L(M)} = \emptyset$, thus $L(M) = \Sigma^*$. Therefore, the Turing machine correctly decides ALL_{DFA}.

13 Question 13

Constructing $L(M) = L(A)\Delta L(B)$. We know that E_{DFA} is in **P**, thus we can construct a Turing machine to decide $L(M) = \emptyset$ in polynomial time. If $L(M) = \emptyset$, then $L(A) = L(B)$; otherwise, $L(A) \neq L(B)$. Therefore, we can decide EQ_{DFA} in polynomial time.